

S7 Control Centre — Live End-to-End Demo Guide

Requirement PDF → live AI epic & stories → real GitHub repository → published delivery packs → developer implements with their own AI CLI.

Every step below was executed and verified live on 9 Aug 2026 as run **S7-00043**, producing the real repository `github.com/AlanLands/advisor-portal-signin`.

Part A — One-time preparation

- Put your model API key in `.env` (e.g. `OPENAI_API_KEY=...`, `LLM_PROVIDER=openai`). Never commit it.
- Authenticate GitHub CLI: `gh auth status` must show your account logged in.
- Start the server in **record** mode so every live call is also recorded for deterministic replay on demo day (hard rule 5):

```
lsof -ti :8720 | xargs kill # stop any stale server
LLM_MODE=record ./demo/run_control.sh
```

- Have the requirement document ready: `demo/MapleSure-Advisor-Portal-Login-Requirement.pdf` — a deliberately small scope (one login page, 7 numbered requirements) so the live build stays demo-sized.

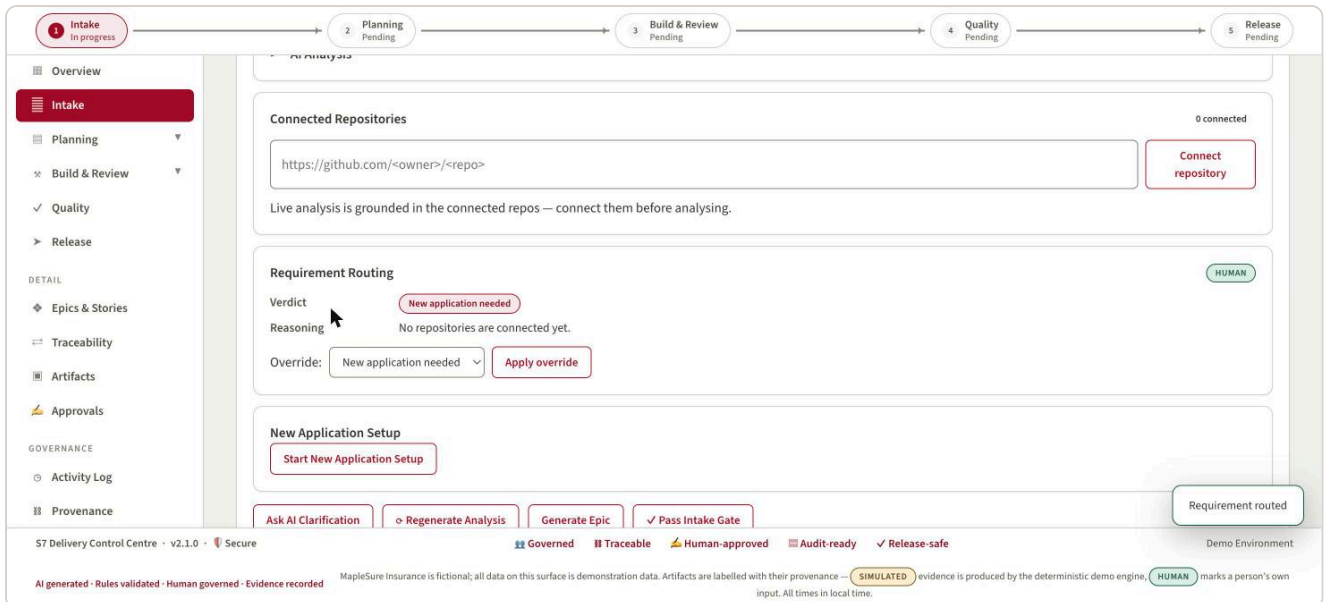
Part B — Intake: PDF in, real repository out VERIFIED LIVE

- Open `http://127.0.0.1:8720`, create a **new run** and set Environment to **Live**. Set your role to **delivery lead** in the header.
- Go to **Intake** → **Upload File** → choose the requirement PDF. Live **AI Extraction** runs automatically and returns the epic title, business objective, summary and REQ-01...REQ-07 — badged **LIVE_AI**.

The screenshot displays the S7 Delivery Control Centre interface. At the top, a progress bar shows five stages: 1. Intake (Ready), 2. Planning (Pending), 3. Build & Review (Pending), 4. Quality (Pending), and 5. Release (Pending). The left sidebar contains a navigation menu with sections: Overview, Intake (selected), Planning, Build & Review, Quality, Release, DETAIL (Epics & Stories, Traceability, Artifacts, Approvals), and GOVERNANCE (Activity Log, Provenance). The main content area is divided into three panels. The left panel shows a file upload section with a 'Browse File' button and supported formats (PDF, DOCX, TXT, MD) with a 10 MB limit. A file named 'MapleSure-Advisor-Portal-Login-Requirement.pdf' is shown as 'Uploaded'. The middle panel displays 'Extracted Requirements' with three items: REQ-1 (sign-in page), REQ-2 (validation), and REQ-3 (password masking). The right panel shows a 'Requirement Summary' and a 'Create Epic & Pass Intake Gate' button. The bottom status bar indicates 'S7 Delivery Control Centre · v2.1.0 · Secure' and includes tags for 'Governed', 'Traceable', 'Human-approved', 'Audit-ready', and 'Release-safe'. It also shows 'Demo Environment' and a provenance statement: 'AI generated · Rules validated · Human governed · Evidence recorded'.

AI Extraction completed from the uploaded PDF — title, objective, summary and numbered requirements, badged **LIVE_AI**.

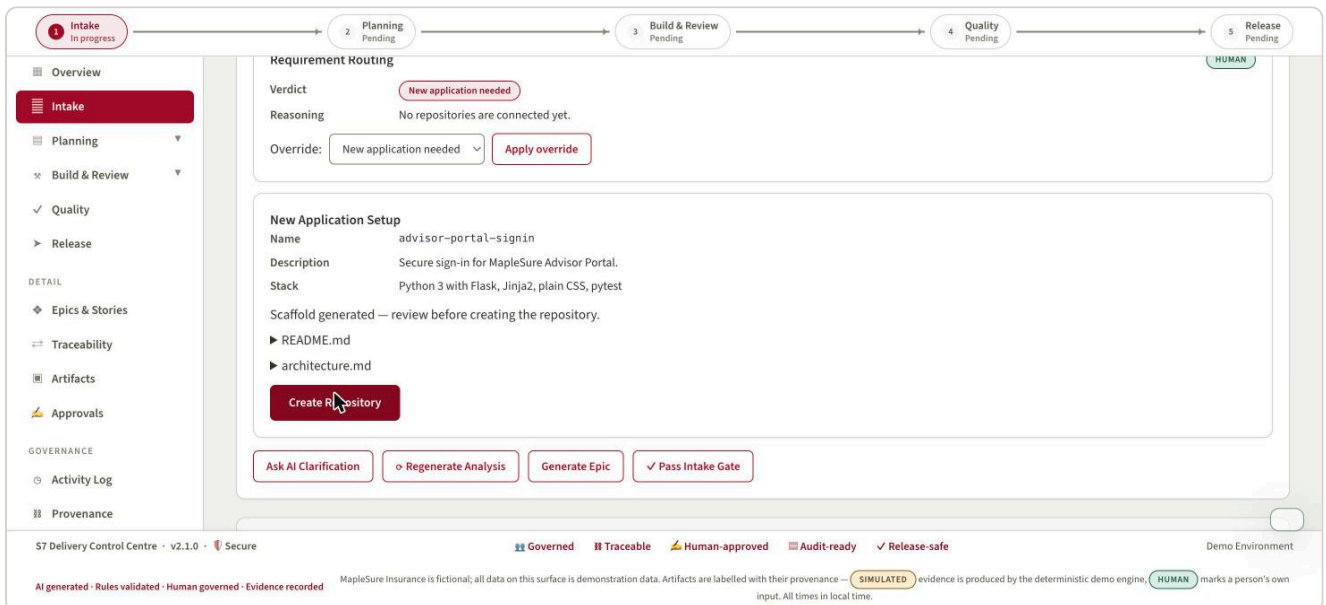
- Expand **Advanced: Live Analysis & Governance** → click **Route Requirement**. With zero repositories connected the verdict short-circuits to **New application needed** (no model call, 100% confidence). A human override dropdown is available.



Requirement Routing: verdict "New application needed" — reasoning stated, override available.

4. Click **Start New Application Setup**. The model asks up to two rounds of questions — ours asked one: the technology stack. Answer (we used: *Python 3 + Flask, Jinja2, plain CSS, pytest, local seeded JSON credential store*) and click **Submit answers**, then click the setup button again to let it finalize. Result: name `advisor-portal-signin`, description and stack settled.

5. Click **Generate Scaffold** — review the proposed `README.md` and `architecture.md` before anything touches GitHub.



Reviewable scaffold; nothing has been created yet — Create Repository is the explicit approval.

6. Click **Create Repository**. This is the real side effect: `gh repo create --push` creates `github.com/<you>/advisor-portal-signin` and connects it to the run like any other repository.

7. Click **Regenerate Analysis** (live AI, now grounded in the new repo), then **Create Epic & Pass Intake Gate**.

Part C — Planning: live stories and Gate 1

VERIFIED LIVE

- Go to **Planning** → **Generate Plan**. Live AI produced 4 stories against the new repository:

Story	Title	Team
US-1	Create Sign-In Page	Portal Team
US-2	Implement Input Validation	Portal Team
US-3	Authenticate Credentials Locally	Platform Team
US-4	Implement Sign-Out Process	Platform Team

- Review the stories, then **Sign off Gate 1** as business owner. The plan locks at v1 and downstream generation is authorised (never performed) by this approval.

Honesty note: any estimates/story points shown are model estimates, not calibrated from historical delivery data — say so if asked.

Part D — Build & Review: publish governed context to Git

VERIFIED LIVE

- Build & Review** → **Architecture**: click **Generate**. The five-file architecture pack v1 is produced with 9/9 validations passing; click **Accept Architecture** (requires the **engineering lead** role).
- Delivery Packs** → **Generate**: one pack per team (Portal, Platform), layered — task packs reference plan/architecture by version.
- Publish All Packs to Git**. In a live run this writes `AGENTS.md` + `.s7/**` (only those paths) on fresh branches `s7/<run>-<team>` and pushes them to the real repository. Never the default branch; nothing is merged for you. Ours pushed as PUB-001/PUB-002 with `simulated = false`.
- Developer Workspaces**: one workspace per story was provisioned. Click **Assign** on your story and enter the developer's name exactly as they will identify themselves (we used *Alan Lands* on US-1).

US-1 assigned to Alan Lands; unassigned rows keep an Assign action.

- Assignment refreshes the team pack to v2 — **Delivery Packs** now shows **UPDATE PENDING** ("v1 on branch — publish v2 to sync assignments"). Click **Republish to Git**. The branch then carries `.s7/shared/assigned-stories.json` with `"assigned_to": "Alan Lands"`.

Intake
Completed

Planning
Completed

Build & Review
Ready

Quality
Pending

Release
Pending

RUN

Overview

Intake

Planning

Build & Review

Overview

Architecture

Delivery Packs

Developer Workspaces

Build & Test Evidence

Independent Review

Build Summary

Quality

Build & Review · Delivery Packs

Delivery Packs

Governed engineering context generated for each team and ready to publish or download.

Download All Packs

Publish All Packs to Git

2 Teams
All teams covered

2 Delivery Packs
All published

4 Stories Covered
100% of scoped stories

52 Artifacts Linked
Across all packs

25 KB Total Pack Size
Uncompressed

0 Publish Errors
All good

Search by team, repository or stories...

ARTIFACT STATUS: All

PUBLICATION STATUS: All

TEAM: All

Reset

TEAM	STORIES	REPOSITORY	PACK VERSION	ARTIFACT STATUS	PUBLICATION STATUS	LAST UPDATED	ACTIONS
PT Portal Team 2 stories	US-1, US-2	advisor-portal-signin service	v2.0 arch v1	CURRENT 26 artifacts	PUBLISHED Synced to Git	09 Aug 2026 13:03	Preview Download
PT Platform Team 2 stories	US-3, US-4	advisor-portal-signin service	v1.0 arch v1	CURRENT 26 artifacts	PUBLISHED Synced to Git	09 Aug 2026 13:02	Preview Download

Showing 2 of 2 packs

Selected Pack

Portal Team Delivery Pack v2.0

REPOSITORY

advisor-portal-signin

PUBLICATION STATUS

PUBLISHED Synced to Git

ARTIFACTS SIZE

26 13 KB

STORIES INCLUDED

US-1 US-2

PACK CONTENTS

Architecture Reference v1 (by reference)

AGENTS.md (Team Guidance) 1

Stories 2

Acceptance Criteria 4

Task Packs 2

Dependencies 1

Test Strategy 1

S7 Delivery Control Centre · v2.1.0 · Secure

Governed Traceable Human-approved Audit-ready Release-safe

Demo Environment

AI generated · Rules validated · Human governed · Evidence recorded

MapleSure Insurance is fictional; all data on this surface is demonstration data. Artifacts are labelled with their provenance — SIMULATED evidence is produced by the deterministic demo engine. HUMAN marks a person's own input. All times in local time.

Both packs PUBLISHED — Synced to Git (real push); Portal Team at v2.0 after the assignment republish.

Part E — The developer machine: any AI CLI, no custom plugin

VERIFIED LIVE

On a fresh machine (nothing installed from S7 — this is the point):

1. Clone and check out the published context branch:

```
git clone https://github.com/<you>/advisor-portal-signin.git
cd advisor-portal-signin
git checkout s7/s7-00043-portal-team
```

The clone contains `AGENTS.md` at the root plus the full `.s7/` tree: shared architecture and engineering rules, per-story packs (story.md, acceptance-criteria.md), per-task packs (task.md, context.json, test-plan.md), and `assigned-stories.json`.

2. Open your coding agent in the clone (Claude Code: just run `claude`) — it reads `AGENTS.md` automatically, which tells it exactly how to answer the next question.
3. Ask: *"What user story is assigned to me? I'm Alan Lands."*
The agent matches your name in `.s7/shared/assigned-stories.json` and answers: **US-1 — Create Sign-In Page** (2 acceptance criteria), then walks you into `.s7/stories/US-1/`. If no story carries your name it says so — the instructions forbid guessing.
4. Say: *"Implement it."* The agent builds the Flask login page live, guided by the acceptance criteria and test plan, staying inside the pack's allowed components. Run it and show the page:

```
pip install flask pytest
python app.py          # open http://127.0.0.1:5000 — the live login page
pytest                 # acceptance-criteria tests
```

5. Commit on a working branch and open a PR. **Merging is a human action** — S7 never merges for you, the same discipline as every other gate.

Why this framing wins the room: S7 is the governed control plane — it generated, validated and published the engineering context. The developer's own AI tooling (any agent that reads the open `AGENTS.md` convention) consumes it. Human Controlled · AI Assisted, with no custom IDE plugin to install or trust.

What is real vs. what is staged in this demo

Real (live)	Staged / simulated (badged in the UI)
PDF extraction, routing, new-app setup, analysis, story planning (all model calls, badged LIVE_AI and recorded for replay)	Build & test evidence, CI status and independent-review execution remain simulation-badged — evidence loop-back from the developer machine is not built yet
GitHub repository creation, branch pushes, assignment sync (verifiable on github.com)	

Demo-day reliability (hard rule 5): this rehearsal ran with `LLM_MODE=record`, so all S7-side model calls are now recorded. On demo day, run with `LLM_MODE=replay` to make the control-plane half deterministic, and keep only the developer-side code generation truly live.